03 — Quy trình làm việc

Cập nhật 03/11/2026 14:30 UTC (thêm §5–§7 từ khung chat 03/11). Giữ tinh thần quy trình gốc (`docs/RESEARCH_WORKFLOW_original.pdf`, `git show de4360e:handoff/04_RESEARCH_WORKFLOW.md`), đổi đích từ "paper" sang "điểm PA tối đa trên private test, trong 6 giờ, tái lập được".

1. Nguyên tắc chung

- **Tự động hoá tối đa.** Tự đề xuất → chạy → kiểm chứng → quyết định; không hỏi lại những quyết định đã có trong tài liệu. Chỉ hỏi user việc chỉ user làm được (token, nộp portal, accept rules, xoá kernel bị chặn, quyền public dữ liệu).
- **Mỗi vòng lặp:** Đề xuất → Phản biện → Xây → Chạy trên Kaggle → Chấm (`src/score.py`) → Phân tích lỗi theo nhóm → Giữ / sửa / bỏ → ghi `EXPERIMENT_LOG.md`.
- **Một thí nghiệm quyết định mỗi lần**, nhưng khi đã chắc thì chạy song song tối đa trên mọi GPU.
- **Đo lỗi trước khi sửa:** luôn lưu đầu ra từng câu (plan, code, kết quả thực thi, đáp án, p) để phân tích offline; tìm vị trí và điều kiện lỗi rồi mới đề xuất bản sửa.
- **Chọn siêu tham số bằng dữ liệu tách riêng** (holdout theo doanh nghiệp), một cấu hình chung; mọi biến thể chọn sau khi nhìn dữ liệu phải được ghi rõ và kiểm lại trên holdout.
- **Báo cáo kết quả âm trung thực**; ý tưởng không cải thiện thì loại và ghi lý do.
- **Kiểm dữ liệu đầu vào trước khi kết luận** (header Excel, định dạng số/ngày, mã hoá tiếng Việt).
- **Không vi phạm luật BTC:** không dùng test để train, không logic theo id, tổng tham số ≤ 15B, offline.

1a. Cổng kiểm tra trước khi chạy (user chốt 2026-11-02)

Code xong → tự review diff (logic, công thức, biên, tối ưu) → `automation/preflight.py "<jobs>"` in `PREFLIGHT OK` (flag, file kernel, test CPU, chạy thử end-to-end trên CPU với mô hình giả) → mới được đẩy kernel. `fanout.py` tự kiểm và từ chối khởi chạy nếu chưa qua. Chi tiết: `CLAUDE.md`.

1b. Nguyên tắc dùng Kaggle (ZAIC.docx §4, cập nhật 2026-11-02)

- Dùng **tất cả** acc được cấp. Token chỉ ở `~/.kaggle/tok_<username>` (quyền 600), không đưa vào code, prompt, GitHub hay báo cáo; kiểm tra kết nối bằng `kaggle competitions list`. Khung chat mới phải xin lại token.
- Acc (username Kaggle, 03/11): L4 ở `lelynhan`, `linhmai24`, `thnliu`, `duylnng`, `caotienminh1`, `nguynduylm`, `phuongnehe`, `hai1108`, `mucnokieu`, `huyenco`; `mxuanquy` chỉ T4; `nguyenmanh0404` chưa accept rules `gemma-4-developer-agent`. **Quota 30 giờ GPU / tuần / acc** (caotienminh1 hết 03/11 14:02); một bản thử đầy đủ (train + 3 public) ≈ 8 giờ GPU. Trạng thái acc mới nhất: `handoff/ENVIRONMENT_SUMMARY.md`. Các acc này **dùng chung với dự án khác** (kernel `e27*`, `e32*`, `nclt-*`…): không đụng vào kernel không phải `zc-*`. Không có token `nhatlong1103` cho kernel (chỉ đăng dataset).
- **Dồn luồng lên một acc**: mỗi acc tối đa 2 phiên GPU + 5 phiên CPU cùng lúc; hết slot mới sang acc khác. Mỗi phiên GPU = 4×L4 (24 GB/GPU, 96 GB phân tán, không phải một GPU 96 GB); không có L4 → 2×T4.
- **Kẹt hàng đợi:** không có đầu ra sau ~8 phút (L4 / CPU) hoặc ~10 phút (T4) → kiểm trạng thái; L4 kẹt hàng đợi → ngắt, tạo phiên mới, tối đa 5 lần; vẫn hỏng → xem lỗi code / logic, sửa rồi chạy lại; không phải lỗi code → chuyển 2×T4. Vẫn hỏng → xem log, quota, cấu hình.
- Tận dụng hết GPU trong phiên: inference `--dp <s GPU>` (mỗi GPU một bản mô hình, chia DN cân bằng theo số câu, prefix caching theo DN); mô hình không vừa 1 GPU thì `--tp`. Fine-tune: DDP mọi GPU, bf16 (L4) / fp16 (T4), gradient accumulation, gradient checkpointing chống OOM.
- Hết GPU: việc rất nhẹ chạy CPU, việc nặng đợi GPU. Định kỳ dọn tiến trình treo (kernel Kaggle và tiến trình nền cục bộ); không ngắt được thì báo user.
- Cần thêm acc (và Kaggle không chặn API) → báo user.

2. Tối ưu thời gian chạy

- Cache mọi thứ tất định (ingest, profile, entity index) và đăng thành dataset public để acc nào cũng mount.
- Lưu đầu ra LLM theo câu → thay đổi ở phần quyết định / formatter / voting chấm lại **offline trong vài giây**, không chạy lại LLM.
- Chia câu hỏi theo doanh nghiệp ra nhiều kernel / GPU; trong kernel dùng vLLM batch + prefix caching.

- Thử nhỏ (vài doanh nghiệp) trước khi chạy toàn bộ.

3. Tối ưu token cho Claude

- **Không chủ động kiểm tra kernel.** Sau khi khởi chạy, bật một watcher nền:

`bash python3 automation/watch_done.py <runs> --max 100`

chạy bằng Bash `run_in_background`. Nó thoát (→ Claude được báo ngay) khi **bất kỳ** kernel nào xong, in tên kernel và trạng thái; xử lý xong thì bật lại. Hết 100 phút thì in `WATCH_TIMEOUT` để bật lại (tác vụ nền có trần ~2 giờ). - **Tìm thông tin:** chỉ đọc văn bản / paper (alphaXiv, web), **không đọc code repo của người khác**. - **Dữ liệu lớn:** chỉ đọc cấu trúc và vài dòng (`head`, `python -c` in shape/cột), không in cả file. - Log dài: chỉ đọc tóm tắt (`score.py` theo nhóm, `--errors N`).

4. Git

- Branch làm việc: `claude/datamate-v3-0dny90` (repo `lhaochen118-ship-it/ZC`). Không tạo PR nếu user không yêu cầu. Chỉ tạo file nộp thử khi kết quả tốt (user chốt).
- Commit kèm trailer theo hướng dẫn của phiên; không ghi model ID vào commit.
- Kết quả mới để file mới (`results/<run>/…`), không ghi đè kết quả cũ.
- Không commit token, dữ liệu sinh ra lớn, cache (`.gitignore`).

5. Vòng lặp hiện tại (từ khi có điểm portal, 03/11)

Train LOO (0.98–0.99) **không còn dự báo** điểm public (0.63–0.69). Vòng lặp đổi thành:

1. **Tìm lỗi chung không cần nhãn** trên log public của bản gần nhất (công cụ: `automation/jobs/`): luật vs LLM bất đồng hoàn toàn (`rule_vs_llm.py`); ánh xạ hạng 1 vs hạng 2 cho đáp án khác (`alt_diff.py`); đồng thuận 3 lần chạy theo DN (`agree_biz.py`); đáp án 0 bất thường; đối chiếu bảng chuẩn với README / rulebook (`two_maps.py`, `scan_maps.py`). Không gán nhãn tay đáp án public.
2. **Sửa bằng quy tắc tổng quát** (luật đọc câu, kiểm tra nội dung ánh xạ, toolkit) + test; kiểm **oracle train 2967 / 1 / 32** và **0 flagged** trên ánh xạ tham chiếu (`tests/eval_lib.py`, `tests/test_plausibility.py`).
3. **Gộp các sửa thành một đợt**, preflight một lần (mỗi lần sửa sau preflight phải chạy lại ~10–12 phút), chạy bản mới (train + 3 public) trên 2 acc; khi chờ thì làm bước 1 cho lỗi tiếp theo.
4. **Đánh giá bản mới bằng so sánh câu-theo-câu với bản đã có điểm portal** (giống / đổi / thêm abstain / bớt abstain theo DN) — không chỉ bằng điểm train. Bản thêm nhiều abstain ở DN lớn thường tụt điểm (v20).
5. File nộp: `automation/map_penalties.py` → `src/vote.py` (tie-break theo ánh xạ) → `automation/make_submission.py` → `submissions/vNN/` + README → **push ngay** → gửi user; user nộp portal và báo điểm → ghi `EXPERIMENT_LOG.md`.

6. Nhiễu của điểm public

- Các lần chạy cùng cấu hình chỉ giống nhau 80–89 % câu (mẫu ánh xạ LLM không tái lập). Điểm public dao động **±0.03** chỉ vì may rủi ánh xạ → một lần nộp không đủ để kết luận một sửa đổi tốt / xấu nếu chênh lệch < 0.03.
- Ưu tiên các sửa giảm phương sai (ổn định ánh xạ, bỏ phiếu biết ánh xạ) trước các sửa tăng trung bình nhỏ.

7. Chuyển khung chat

- Trước khi đóng: commit + push mọi thứ; chép job / script khỏi scratch vào repo (`automation/jobs/`); cập nhật `handoff/STATUS.md`, `EXPERIMENT_LOG.md`, ba PDF bàn giao (`handoff/0[1-3]_*.pdf`), `handoff/environment_summary.pdf`; tắt routine tự chạy.
- Khung mới: đọc `READING_GUIDE.md` → `CLAUDE.md` → `handoff/STATUS.md`; xin token; tải lại output kernel đang chạy.